

Sri Lanka Institute of Information Technology



Web-based Help Desk for University Students

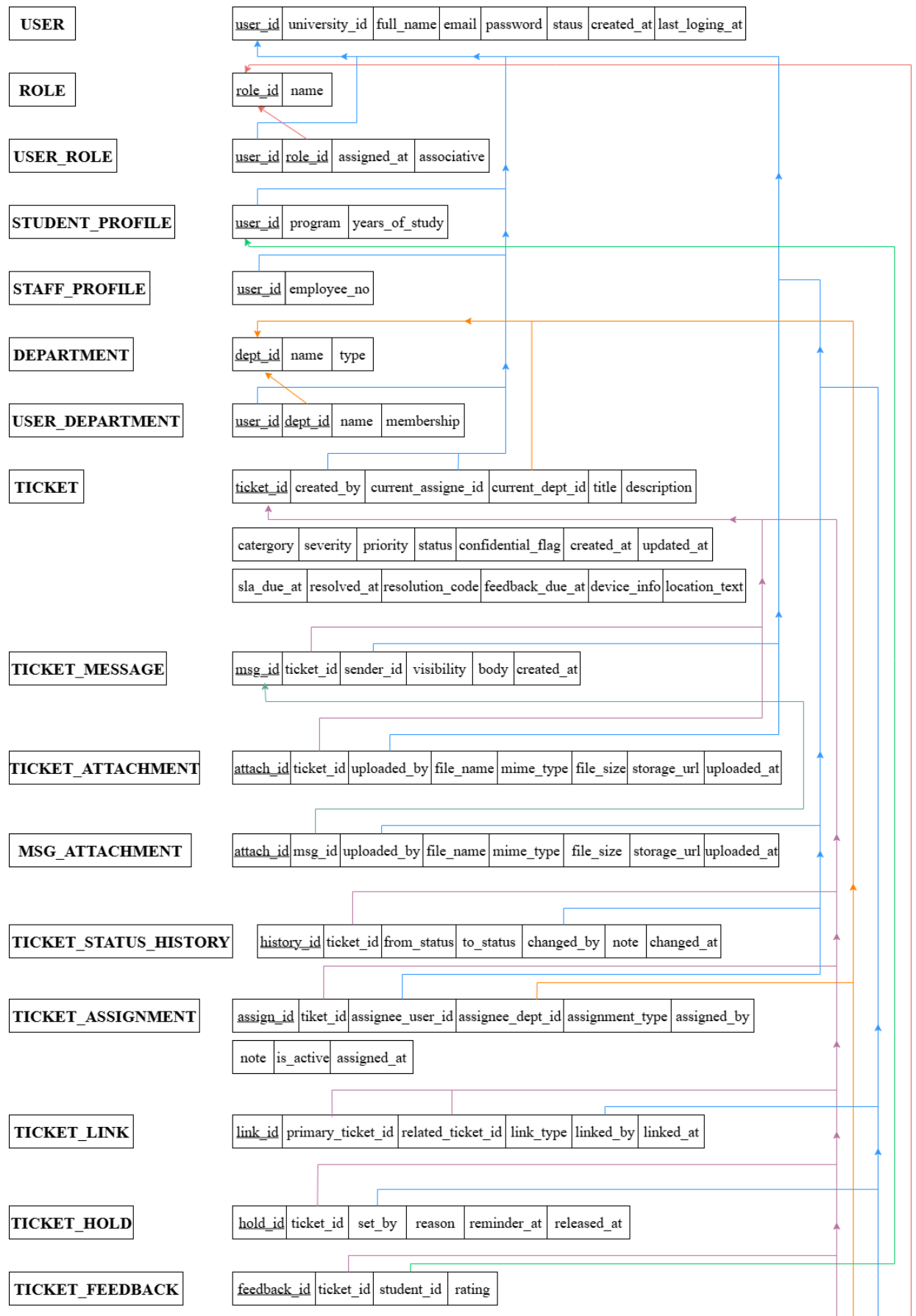
Year 2 – Semester 1 2025

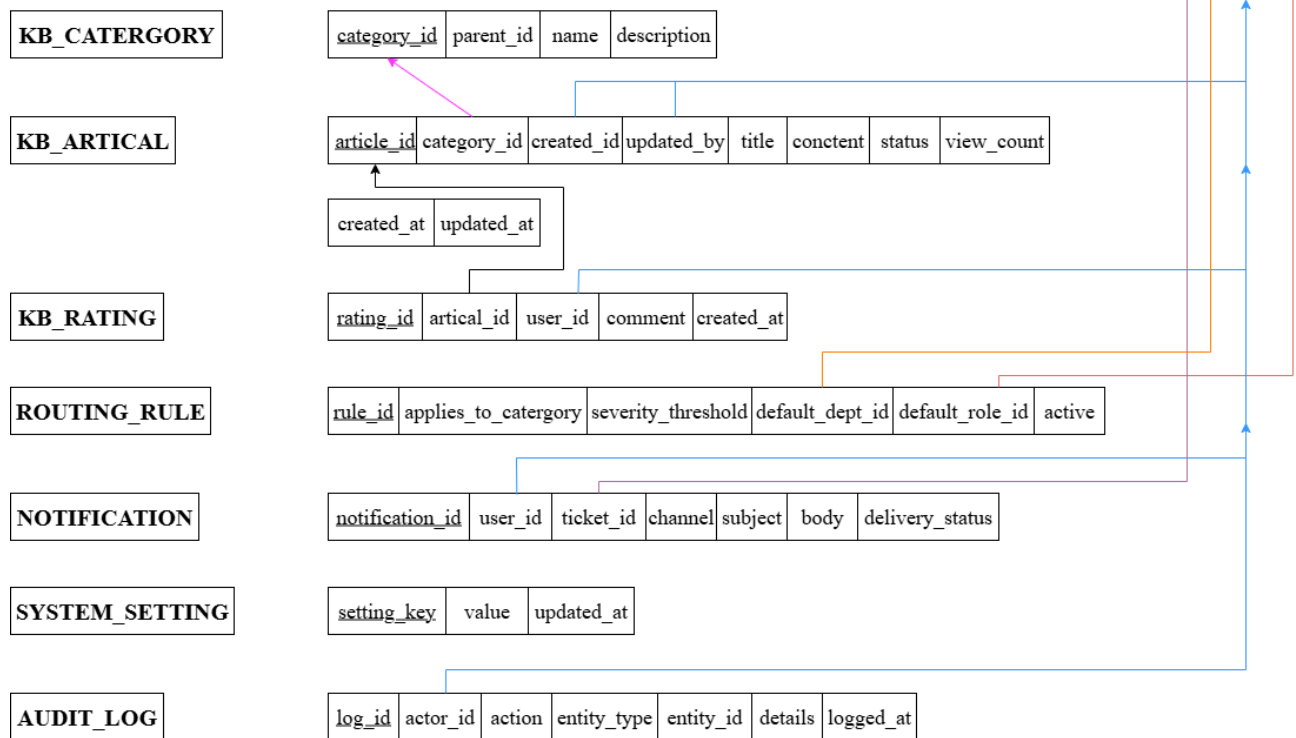
MLB-B2G2-10	
IT24100197	Rosa S.H.V.A
IT24100151	Ruth R W N
IT24100078	Ilesingha I.Y.R
IT24100212	Hanwellage P.M
IT24100139	Pinthu D.I.U.
IT24100186	Yeshan K.P

Database Design and Development | IT2140

B.Sc. (Hons) in Information Technology

1. Part A – Mapping EER to Relational Schema





ISA decision (USER $\rightarrow$ STUDENT_PROFILE, STAFF_PROFILE)

- **Specialization semantics assumed: partial, overlapping** (a USER may be neither, or both, a student and staff).
- **Chosen option: ISA Option 1** — one table for the supertype (USER) and one table per subtype with the same PK also acting as FK to USER. This option cleanly supports any combo of disjoint/overlapping and total/partial constraints and avoids wide, sparse “single table” designs. It also lets us enforce “only students can do X” by FK’ing to the subtype table.

2. Part B – SQL DDL Implementation

```
Part B.sql - DESKT...Q5FRH3\isu23 (74))* X SQLQuery1.sql - DE...Q5FRH3\isu23 (66))*
-- IT2140 DDD - Assignment 01 Part 02 (Part B: SQL DDL)

-- -----
-- Core reference tables
-- -----

CREATE TABLE [user] (
    user_id INT IDENTITY(1,1) PRIMARY KEY,
    university_id VARCHAR(20) UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    last_login_at DATETIME2 NULL,
    CONSTRAINT chk_user_status CHECK (status IN ('ACTIVE', 'INACTIVE', 'SUSPENDED'))
);

CREATE TABLE [role] (
    role_id INT IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE department (
    dept_id INT IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    type VARCHAR(20) NOT NULL,
    CONSTRAINT chk_dept_type CHECK (type IN ('ACADEMIC', 'ADMIN', 'IT', 'OTHER'))
);

-- ISA (Option-1): subtype PK = FK to [user]
CREATE TABLE student_profile (
    user_id INT PRIMARY KEY,
    program VARCHAR(80) NOT NULL,
    year_of_study INT NOT NULL,
    CONSTRAINT FK_student_profile_user
        FOREIGN KEY (user_id) REFERENCES [user](user_id) ON DELETE CASCADE,
    CONSTRAINT chk_student_year CHECK (year_of_study >= 1)
);

CREATE TABLE staff_profile (
    user_id INT PRIMARY KEY,
    employee_no VARCHAR(30) NOT NULL UNIQUE,
    CONSTRAINT FK_staff_profile_user
        FOREIGN KEY (user_id) REFERENCES [user](user_id) ON DELETE CASCADE
);
```

```

CREATE TABLE user_role (
    user_id INT NOT NULL,
    role_id INT NOT NULL,
    assigned_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    PRIMARY KEY (user_id, role_id),
    CONSTRAINT FK_user_role_user FOREIGN KEY (user_id) REFERENCES [user](user_id) ON DELETE CASCADE,
    CONSTRAINT FK_user_role_role FOREIGN KEY (role_id) REFERENCES [role](role_id) ON DELETE CASCADE
);

CREATE TABLE user_department (
    user_id INT NOT NULL,
    dept_id INT NOT NULL,
    PRIMARY KEY (user_id, dept_id),
    CONSTRAINT FK_user_dept_user FOREIGN KEY (user_id) REFERENCES [user](user_id) ON DELETE CASCADE,
    CONSTRAINT FK_user_dept_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id) ON DELETE CASCADE
);

```

```

-- Tickets
CREATE TABLE ticket (
    ticket_id INT IDENTITY(1,1) PRIMARY KEY,
    created_by INT NOT NULL,
    current_assignee_id INT NULL,
    current_dept_id INT NULL,
    title VARCHAR(200) NOT NULL,
    description VARCHAR(MAX) NOT NULL,
    category VARCHAR(50) NOT NULL,
    severity VARCHAR(10) NOT NULL,
    priority VARCHAR(5) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
    confidential_flag BIT NOT NULL DEFAULT 0,
    created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    updated_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    sla_due_at DATETIME2 NULL,
    resolved_at DATETIME2 NULL,
    resolution_code VARCHAR(50) NULL,
    feedback_due_at DATETIME2 NULL,
    device_info VARCHAR(200) NULL,
    location_text VARCHAR(200) NULL,
    CONSTRAINT FK_ticket_created_by FOREIGN KEY (created_by) REFERENCES [user](user_id),
    CONSTRAINT FK_ticket_assignee FOREIGN KEY (current_assignee_id) REFERENCES [user](user_id) ON DELETE SET NULL,
    CONSTRAINT FK_ticket_dept FOREIGN KEY (current_dept_id) REFERENCES department(dept_id) ON DELETE SET NULL,
    CONSTRAINT chk_ticket_severity CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
    CONSTRAINT chk_ticket_priority CHECK (priority IN ('P3', 'P2', 'P1')),
    CONSTRAINT chk_ticket_status CHECK (status IN ('OPEN', 'IN_PROGRESS', 'ON_HOLD', 'RESOLVED', 'CLOSED'))
);

```

```

CREATE TABLE ticket_message (
    message_id INT IDENTITY(1,1) PRIMARY KEY,
    ticket_id INT NOT NULL,
    sender_id INT NOT NULL,
    visibility VARCHAR(10) NOT NULL,
    body VARCHAR(MAX) NOT NULL,
    created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_msg_ticket FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id) ON DELETE CASCADE,
    CONSTRAINT FK_msg_sender FOREIGN KEY (sender_id) REFERENCES [user](user_id),
    CONSTRAINT chk_msg_visibility CHECK (visibility IN ('PUBLIC', 'INTERNAL'))
);

```

```

CREATE TABLE ticket_attachment (
    attachment_id INT IDENTITY(1,1) PRIMARY KEY,
    ticket_id INT NOT NULL,
    uploaded_by INT NOT NULL,
    file_name VARCHAR(255) NOT NULL,
    mime_type VARCHAR(100) NOT NULL,
    file_size INT NOT NULL CHECK (file_size >= 0),
    storage_url VARCHAR(400) NOT NULL,
    uploaded_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_tattach_ticket FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id) ON DELETE CASCADE,
    CONSTRAINT FK_tattach_uploader FOREIGN KEY (uploaded_by) REFERENCES [user](user_id)
);

```

```

CREATE TABLE message_attachment (
    attachment_id INT IDENTITY(1,1) PRIMARY KEY,
    message_id INT NOT NULL,
    uploaded_by INT NOT NULL,
    file_name VARCHAR(255) NOT NULL,
    mime_type VARCHAR(100) NOT NULL,
    file_size INT NOT NULL CHECK (file_size >= 0),
    storage_url VARCHAR(400) NOT NULL,
    uploaded_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_mattach_msg FOREIGN KEY (message_id) REFERENCES ticket_message(message_id) ON DELETE CASCADE,
    CONSTRAINT FK_mattach_uploader FOREIGN KEY (uploaded_by) REFERENCES [user](user_id)
);

```

```

CREATE TABLE ticket_status_history (
    history_id INT IDENTITY(1,1) PRIMARY KEY,
    ticket_id INT NOT NULL,
    from_status VARCHAR(20) NOT NULL,
    to_status VARCHAR(20) NOT NULL,
    changed_by INT NOT NULL,
    note VARCHAR(MAX) NULL,
    changed_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_hist_ticket FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id) ON DELETE CASCADE,
    CONSTRAINT FK_hist_user FOREIGN KEY (changed_by) REFERENCES [user](user_id),
    CONSTRAINT chk_hist_from CHECK (from_status IN ('OPEN', 'IN_PROGRESS', 'ON_HOLD', 'RESOLVED', 'CLOSED')),
    CONSTRAINT chk_hist_to CHECK (to_status IN ('OPEN', 'IN_PROGRESS', 'ON_HOLD', 'RESOLVED', 'CLOSED'))
);

```

```

CREATE TABLE ticket_assignment (
    assign_id INT IDENTITY(1,1) PRIMARY KEY,
    ticket_id INT NOT NULL,
    assignee_user_id INT NULL,
    assignee_dept_id INT NULL,
    assignment_type VARCHAR(15) NOT NULL,
    assigned_by INT NOT NULL,
    note VARCHAR(MAX) NULL,
    is_active BIT NOT NULL DEFAULT 1,
    assigned_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_asg_ticket FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id) ON DELETE CASCADE,
    CONSTRAINT FK_asg_user FOREIGN KEY (assignee_user_id) REFERENCES [user](user_id),
    CONSTRAINT FK_asg_dept FOREIGN KEY (assignee_dept_id) REFERENCES department(dept_id),
    CONSTRAINT FK_asg_by_user FOREIGN KEY (assigned_by) REFERENCES [user](user_id),
    -- Exactly one assignee provided (user XOR dept)
    CONSTRAINT chk_assign_xor CHECK (
        (CASE WHEN assignee_user_id IS NULL THEN 0 ELSE 1 END) +
        (CASE WHEN assignee_dept_id IS NULL THEN 0 ELSE 1 END) = 1
    ),
    CONSTRAINT chk_assign_type CHECK (assignment_type IN ('DIRECT', 'AUTO', 'ESCALATION'))
);

```

```

CREATE TABLE ticket_link (
    link_id INT IDENTITY(1,1) PRIMARY KEY,
    primary_ticket_id INT NOT NULL,
    related_ticket_id INT NOT NULL,
    link_type VARCHAR(20) NOT NULL,
    linked_by INT NOT NULL,
    linked_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_link_primary FOREIGN KEY (primary_ticket_id) REFERENCES ticket(ticket_id),
    CONSTRAINT FK_link_related FOREIGN KEY (related_ticket_id) REFERENCES ticket(ticket_id),
    CONSTRAINT FK_link_user FOREIGN KEY (linked_by) REFERENCES [user](user_id),
    -- prevent self-link and enforce canonical order for pairs
    CONSTRAINT chk_link_not_self CHECK (primary_ticket_id <> related_ticket_id),
    CONSTRAINT chk_link_order CHECK (primary_ticket_id < related_ticket_id)
);

-- With the order check above, this guarantees uniqueness with a simple unique index:
CREATE UNIQUE INDEX uq_ticket_link_pair ON ticket_link(primary_ticket_id, related_ticket_id);

CREATE TABLE ticket_hold (
    hold_id INT IDENTITY(1,1) PRIMARY KEY,
    ticket_id INT NOT NULL,
    set_by INT NOT NULL,
    reason VARCHAR(200) NOT NULL,
    reminder_at DATETIME2 NULL,
    released_at DATETIME2 NULL,
    CONSTRAINT FK_hold_ticket FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id) ON DELETE CASCADE,
    CONSTRAINT FK_hold_user FOREIGN KEY (set_by) REFERENCES [user](user_id)
);

CREATE TABLE ticket_feedback (
    feedback_id INT IDENTITY(1,1) PRIMARY KEY,
    ticket_id INT NOT NULL UNIQUE,
    student_id INT NOT NULL,
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment VARCHAR(MAX) NULL,
    submitted_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_fb_ticket FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id) ON DELETE CASCADE,
    CONSTRAINT FK_fb_student FOREIGN KEY (student_id) REFERENCES student_profile(user_id)
);

-- -----
-- Knowledge Base
-- -----

CREATE TABLE kb_category (
    category_id INT IDENTITY(1,1) PRIMARY KEY,
    parent_id INT NULL,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(MAX) NULL,
    CONSTRAINT FK_kbcat_parent
        FOREIGN KEY (parent_id) REFERENCES kb_category(category_id), -- NO CASCADE
    CONSTRAINT uq_kbcat_parent_name UNIQUE (parent_id, name),
    CONSTRAINT chk_kbcat_self CHECK (parent_id IS NULL OR parent_id <> category_id)
);

```

```

CREATE TABLE kb_article (
    article_id INT IDENTITY(1,1) PRIMARY KEY,
    category_id INT NOT NULL,
    created_by INT NOT NULL,
    updated_by INT NOT NULL,
    title VARCHAR(200) NOT NULL,
    content VARCHAR(MAX) NOT NULL,
    status VARCHAR(12) NOT NULL,
    view_count INT NOT NULL DEFAULT 0 CHECK (view_count >= 0),
    created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    updated_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_kb_art_cat FOREIGN KEY (category_id) REFERENCES kb_category(category_id),
    CONSTRAINT FK_kb_art_cby FOREIGN KEY (created_by) REFERENCES [user](user_id),
    CONSTRAINT FK_kb_art_uby FOREIGN KEY (updated_by) REFERENCES [user](user_id),
    CONSTRAINT chk_kb_status CHECK (status IN ('DRAFT', 'PUBLISHED', 'ARCHIVED'))
);

CREATE TABLE kb_rating (
    rating_id INT IDENTITY(1,1) PRIMARY KEY,
    article_id INT NOT NULL,
    user_id INT NOT NULL,
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment VARCHAR(MAX) NULL,
    created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_kbr_article FOREIGN KEY (article_id) REFERENCES kb_article(article_id) ON DELETE CASCADE,
    CONSTRAINT FK_kbr_user FOREIGN KEY (user_id) REFERENCES [user](user_id) ON DELETE CASCADE,
    CONSTRAINT uq_kb_rating UNIQUE (article_id, user_id)
);

-- -----
-- Routing / Notifications / Settings / Audit
-- -----
CREATE TABLE routing_rule (
    rule_id INT IDENTITY(1,1) PRIMARY KEY,
    applies_to_category VARCHAR(50) NOT NULL,
    severity_threshold VARCHAR(10) NOT NULL,
    default_dept_id INT NOT NULL,
    default_role_id INT NOT NULL,
    active BIT NOT NULL DEFAULT 1,
    CONSTRAINT FK_rr_dept FOREIGN KEY (default_dept_id) REFERENCES department(dept_id),
    CONSTRAINT FK_rr_role FOREIGN KEY (default_role_id) REFERENCES [role](role_id),
    CONSTRAINT chk_routing_severity CHECK (severity_threshold IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'))
);

CREATE TABLE notification (
    notification_id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL,
    ticket_id INT NULL,
    channel VARCHAR(10) NOT NULL,
    subject VARCHAR(200) NOT NULL,
    body VARCHAR(MAX) NOT NULL,
    delivery_status VARCHAR(10) NOT NULL DEFAULT 'QUEUED',
    sent_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_notif_user FOREIGN KEY (user_id) REFERENCES [user](user_id) ON DELETE CASCADE,
    CONSTRAINT FK_notif_ticket FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id) ON DELETE SET NULL,
    CONSTRAINT chk_notif_channel CHECK (channel IN ('EMAIL', 'SMS', 'PUSH', 'INAPP')),
    CONSTRAINT chk_notif_status CHECK (delivery_status IN ('QUEUED', 'SENT', 'FAILED', 'READ'))
);

CREATE TABLE system_setting (
    setting_key VARCHAR(100) PRIMARY KEY,
    value VARCHAR(MAX) NOT NULL,
    updated_at DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);

```



```

CREATE TABLE audit_log (
    log_id INT IDENTITY(1,1) PRIMARY KEY,
    actor_id INT NULL,
    action VARCHAR(50) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id INT NOT NULL,
    details VARCHAR(MAX) NULL,
    logged_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_audit_actor FOREIGN KEY (actor_id) REFERENCES [user](user_id) ON DELETE SET NULL
);

```

SQLQuery1.sql - DE...Q5FRH3\isu23 (64))* Part C.sql - DE

`SELECT name FROM sys.tables ORDER BY name;`

100 %

Results Messages

	name
1	audit_log
2	department
3	kb_article
4	kb_category
5	kb_rating
6	message_attachment
7	notification
8	role
9	routing_rule
10	staff_profile
11	student_profile
12	system_setting
13	ticket
14	ticket_assignment
15	ticket_attachment
16	ticket_feedback
17	ticket_hold
18	ticket_link
19	ticket_message
20	ticket_status_history
21	user
22	user_department
23	user_role

3. Part C – Insert Sample Data

```
-----  
-- 1) Roles  
-----  
INSERT INTO [role](name)  
SELECT v.name  
FROM (VALUES ('STUDENT'),('STAFF'),('AGENT'),('MANAGER')) AS v(name)  
WHERE NOT EXISTS (SELECT 1 FROM [role] r WHERE r.name=v.name);  
  
-----  
-- 2) Users  
-----  
INSERT INTO [user](university_id, full_name, email, password_hash, status)  
SELECT v.university_id, v.full_name, v.email, v.password_hash, 'ACTIVE'  
FROM (VALUES  
    ('U2025-001', 'Nimal Perera', 'nimal@uni.test', 'hash_nimal'),  
    ('U2025-002', 'Sithmi Karun', 'sithmi@uni.test', 'hash_sithmi'),  
    ('E2025-101', 'Kasun Silva', 'kasun@uni.test', 'hash_kasun'),  
    ('E2025-102', 'Tharuni Dias', 'tharuni@uni.test', 'hash_tharuni'),  
    ('E2025-001', 'Manager IT', 'manager.it@uni.test', 'hash_mgr')  
    ) AS v(university_id, full_name, email, password_hash)  
WHERE NOT EXISTS (SELECT 1 FROM [user] u WHERE u.university_id=v.university_id);  
  
-----  
-- 3) ISA subtypes  
-----  
-- Students  
INSERT INTO student_profile (user_id, program, year_of_study)  
SELECT u.user_id, v.program, v.year_of_study  
FROM (VALUES  
    ('U2025-001', 'BSc IT', 2),  
    ('U2025-002', 'BSc CS', 1)  
    ) AS v(university_id, program, year_of_study)  
JOIN [user] u ON u.university_id=v.university_id  
LEFT JOIN student_profile sp ON sp.user_id=u.user_id  
WHERE sp.user_id IS NULL;  
  
-- Staff  
INSERT INTO staff_profile (user_id, employee_no)  
SELECT u.user_id, v.employee_no  
FROM (VALUES  
    ('E2025-101', 'EMP-101'),  
    ('E2025-102', 'EMP-102'),  
    ('E2025-001', 'EMP-001')  
    ) AS v(university_id, employee_no)  
JOIN [user] u ON u.university_id=v.university_id  
LEFT JOIN staff_profile sf ON sf.user_id=u.user_id  
WHERE sf.user_id IS NULL;
```

```
-----  
-- 4) User / Role  
-----
```

```
INSERT INTO user_role (user_id, role_id)  
SELECT u.user_id, r.role_id  
FROM (VALUES  
    ('U2025-001', 'STUDENT'),  
    ('U2025-002', 'STUDENT'),  
    ('E2025-101', 'STAFF'),  
    ('E2025-101', 'AGENT'),  
    ('E2025-102', 'STAFF'),  
    ('E2025-102', 'AGENT'),  
    ('E2025-001', 'STAFF'),  
    ('E2025-001', 'MANAGER')  
 ) AS map(university_id, role_name)  
JOIN [user] u ON u.university_id=map.university_id  
JOIN [role] r ON r.name=map.role_name  
LEFT JOIN user_role ur ON ur.user_id=u.user_id AND ur.role_id=r.role_id  
WHERE ur.user_id IS NULL;
```

```
-----  
-- 5) Departments  
-----
```

```
INSERT INTO department(name,type)  
SELECT v.name, v.type  
FROM (VALUES  
    ('IT Helpdesk', 'IT'),  
    ('Academic Affairs', 'ACADEMIC'),  
    ('Admin Office', 'ADMIN')  
 ) AS v(name, type)  
WHERE NOT EXISTS (SELECT 1 FROM department d WHERE d.name=v.name);
```

```
-- User / Department
```

```
INSERT INTO user_department(user_id, dept_id)  
SELECT u.user_id, d.dept_id  
FROM (VALUES  
    ('E2025-101', 'IT Helpdesk'),  
    ('E2025-102', 'IT Helpdesk'),  
    ('E2025-001', 'IT Helpdesk')  
 ) AS map(university_id, dept_name)  
JOIN [user] u ON u.university_id=map.university_id  
JOIN department d ON d.name=map.dept_name  
LEFT JOIN user_department ud ON ud.user_id=u.user_id AND ud.dept_id=d.dept_id  
WHERE ud.user_id IS NULL;
```

```

-----
-- 6) Tickets
-----
;WITH T AS (
  SELECT *
  FROM (VALUES
    -- created_by, assignee_user (nullable), dept (nullable), title, description, category, severity, priority, status, confidential, device_info, location_text
    ('U2025-001','E2025-101',NULL,'Wi-Fi not working','Cannot connect to campus Wi-Fi since morning','Network','HIGH','P1','IN_PROGRESS',0,'Laptop - Dell','Library L1'),
    ('U2025-002',NULL,'IT Helpdesk','Password reset','Forgot LMS password; need reset','Accounts','MEDIUM','P2','OPEN',0,NULL,NULL),
    ('U2025-001',NULL,'IT Helpdesk','Lab PC broken','PC in Lab-3 not booting','Hardware','HIGH','P1','ON_HOLD',0,'Desktop - HP','Lab 3'),
    ('U2025-002',NULL,'Academic Affairs','Official transcript','Need official transcript for internship','Academic','LOW','P3','RESOLVED',0,NULL,NULL)
  ) AS v(created_by_uid, assignee_uid, dept_name, title, description, category, severity, priority, status, confidential_flag, device_info, location_text)
)
INSERT INTO ticket
(created_by, current_assignee_id, current_dept_id, title, description, category, severity, priority, status, confidential_flag, device_info, location_text)
SELECT
  u_created.user_id,
  u_assignee.user_id,
  d.dept_id,
  T.title, T.description, T.category, T.severity, T.priority, T.status, T.confidential_flag, T.device_info, T.location_text
FROM T
JOIN [user] u_created ON u_created.university_id=T.created_by_uid
LEFT JOIN [user] u_assignee ON u_assignee.university_id=T.assignee_uid
LEFT JOIN department d ON d.name=T.dept_name
WHERE NOT EXISTS (
  SELECT 1 FROM ticket t
  WHERE t.title = T.title AND t.created_by = u_created.user_id
);

-----
-- 7) Ticket messages
-----
INSERT INTO ticket_message(ticket_id, sender_id, visibility, body)
SELECT t.ticket_id, u.user_id, v.visibility, v.body
FROM (VALUES
  ('Wi-Fi not working','U2025-001','PUBLIC', 'I tried eduroam and the guest network; neither works.'),
  ('Wi-Fi not working','E2025-101','INTERNAL','Checking AP status near Library L1.'),
  ('Password reset', 'U2025-002','PUBLIC', 'Forgot password; registered email is sithmi@uni.test'),
  ('Lab PC broken', 'U2025-001','PUBLIC', 'PC says "No boot device".'),
  ('Official transcript','E2025-102','PUBLIC', 'Your transcript has been generated and emailed.')
) AS v(ticket_title, sender_uid, visibility, body)
JOIN [user] u ON u.university_id=v.sender_uid
JOIN ticket t ON t.title=v.ticket_title
JOIN [user] cu ON cu.user_id=t.created_by -- ensure same tickets as inserted
LEFT JOIN ticket_message tm
  ON tm.ticket_id=t.ticket_id AND tm.sender_id=u.user_id AND tm.body=v.body
WHERE tm.message_id IS NULL;

-----
-- 8) One attachment sample
-----
INSERT INTO ticket_attachment(ticket_id, uploaded_by, file_name, mime_type, file_size, storage_url)
SELECT t.ticket_id, u.user_id, 'wifi_screenshot.png', 'image/png', 123456, 'blob://tickets/t1/wifi_screenshot.png'
FROM ticket t
JOIN [user] u ON u.university_id='U2025-001'
WHERE t.title='Wi-Fi not working'
  AND NOT EXISTS (SELECT 1 FROM ticket_attachment a WHERE a.ticket_id=t.ticket_id AND a.file_name='wifi_screenshot.png');

-----
-- 9) Status history
-----
INSERT INTO ticket_status_history(ticket_id, from_status, to_status, changed_by, note)
SELECT t.ticket_id, 'OPEN','IN_PROGRESS', u.user_id, 'Picked up by agent1'
FROM ticket t
JOIN [user] u ON u.university_id='E2025-101'
WHERE t.title='Wi-Fi not working'
  AND NOT EXISTS (SELECT 1 FROM ticket_status_history h WHERE h.ticket_id=t.ticket_id AND h.from_status='OPEN' AND h.to_status='IN_PROGRESS');

INSERT INTO ticket_status_history(ticket_id, from_status, to_status, changed_by, note)
SELECT t.ticket_id, 'OPEN','IN_PROGRESS', u.user_id, 'Assigned to agent2'
FROM ticket t
JOIN [user] u ON u.university_id='E2025-102'
WHERE t.title='Official transcript'
  AND NOT EXISTS (SELECT 1 FROM ticket_status_history h WHERE h.ticket_id=t.ticket_id AND h.from_status='OPEN' AND h.to_status='IN_PROGRESS');

INSERT INTO ticket_status_history(ticket_id, from_status, to_status, changed_by, note)
SELECT t.ticket_id, 'IN_PROGRESS','RESOLVED', u.user_id, 'Completed and notified user'
FROM ticket t
JOIN [user] u ON u.university_id='E2025-102'
WHERE t.title='Official transcript'
  AND NOT EXISTS (SELECT 1 FROM ticket_status_history h WHERE h.ticket_id=t.ticket_id AND h.from_status='IN_PROGRESS' AND h.to_status='RESOLVED');

```

```

-----
-- 10) Assignments (XOR respected; <=1 active per ticket)
-----
-- T1 → agent1 (active)
INSERT INTO ticket_assignment(ticket_id, assignee_user_id, assignee_dept_id, assignment_type, assigned_by, note, is_active)
SELECT t.ticket_id, uA.user_id, NULL, 'DIRECT', uM.user_id, 'Assigned to agent1', 1
FROM ticket t
JOIN [user] uA ON uA.university_id='E2025-101'
JOIN [user] uM ON uM.university_id='E2025-001'
WHERE t.title='Wi-Fi not working'
AND NOT EXISTS (SELECT 1 FROM ticket_assignment a WHERE a.ticket_id=t.ticket_id AND a.assignee_user_id=uA.user_id AND a.is_active=1);

-- T2 → IT dept (active)
INSERT INTO ticket_assignment(ticket_id, assignee_user_id, assignee_dept_id, assignment_type, assigned_by, note, is_active)
SELECT t.ticket_id, NULL, d.dept_id, 'AUTO', uM.user_id, 'Auto-routed by rule', 1
FROM ticket t
JOIN department d ON d.name='IT Helpdesk'
JOIN [user] uM ON uM.university_id='E2025-001'
WHERE t.title='Password reset'
AND NOT EXISTS (SELECT 1 FROM ticket_assignment a WHERE a.ticket_id=t.ticket_id AND a.assignee_dept_id=d.dept_id AND a.is_active=1);

-- T3 → first IT dept (inactive), then agent2 (active)
INSERT INTO ticket_assignment(ticket_id, assignee_user_id, assignee_dept_id, assignment_type, assigned_by, note, is_active)
SELECT t.ticket_id, NULL, d.dept_id, 'AUTO', uM.user_id, 'Initial routing', 0
FROM ticket t
JOIN department d ON d.name='IT Helpdesk'
JOIN [user] uM ON uM.university_id='E2025-001'
WHERE t.title='Lab PC broken'
AND NOT EXISTS (SELECT 1 FROM ticket_assignment a WHERE a.ticket_id=t.ticket_id AND a.assignee_dept_id=d.dept_id AND a.is_active=0);

INSERT INTO ticket_assignment(ticket_id, assignee_user_id, assignee_dept_id, assignment_type, assigned_by, note, is_active)
SELECT t.ticket_id, uA2.user_id, NULL, 'ESCALATION', uM.user_id, 'Escalated to agent2', 1
FROM ticket t
JOIN [user] uA2 ON uA2.university_id='E2025-102'
JOIN [user] uM ON uM.university_id='E2025-001'
WHERE t.title='Lab PC broken'
AND NOT EXISTS (SELECT 1 FROM ticket_assignment a WHERE a.ticket_id=t.ticket_id AND a.assignee_user_id=uA2.user_id AND a.is_active=1);

-- T4 → agent2 (inactive since resolved)
INSERT INTO ticket_assignment(ticket_id, assignee_user_id, assignee_dept_id, assignment_type, assigned_by, note, is_active)
SELECT t.ticket_id, uA2.user_id, NULL, 'DIRECT', uM.user_id, 'Handled and resolved', 0
FROM ticket t
JOIN [user] uA2 ON uA2.university_id='E2025-102'
JOIN [user] uM ON uM.university_id='E2025-001'
WHERE t.title='Official transcript'
AND NOT EXISTS (SELECT 1 FROM ticket_assignment a WHERE a.ticket_id=t.ticket_id AND a.assignee_user_id=uA2.user_id AND a.is_active=0);

-----
-- 11) Ticket link (T1 ↔ T3), enforcing ordered pair (primary < related)
-----
;WITH pair AS (
    SELECT
        (SELECT ticket_id FROM ticket WHERE title='Wi-Fi not working') AS tA,
        (SELECT ticket_id FROM ticket WHERE title='Lab PC broken') AS tB,
        (SELECT user_id FROM [user] WHERE university_id='E2025-101') AS linker
    )
INSERT INTO ticket_link(primary_ticket_id, related_ticket_id, link_type, linked_by)
SELECT
    CASE WHEN tA < tB THEN tA ELSE tB END,
    CASE WHEN tA < tB THEN tB ELSE tA END,
    'RELATES_TO', linker
FROM pair
WHERE NOT EXISTS (
    SELECT 1 FROM ticket_link tl
    WHERE tl.primary_ticket_id = CASE WHEN tA < tB THEN tA ELSE tB END
    AND tl.related_ticket_id = CASE WHEN tA < tB THEN tB ELSE tA END
);

```

```

-----
-- 12) Holds & Feedback
-----
INSERT INTO ticket_hold(ticket_id, set_by, reason, reminder_at)
SELECT t.ticket_id, u.user_id, 'Waiting for spare parts', DATEADD(DAY,2,SYSDATETIME())
FROM ticket t
JOIN [user] u ON u.university_id='E2025-102'
WHERE t.title='Lab PC broken'
AND NOT EXISTS (SELECT 1 FROM ticket_hold h WHERE h.ticket_id=t.ticket_id);

INSERT INTO ticket_feedback(ticket_id, student_id, rating, comment)
SELECT t.ticket_id, u.user_id, 5, 'Quick and helpful!'
FROM ticket t
JOIN [user] cu ON cu.user_id=t.created_by
JOIN [user] u ON u.university_id='U2025-002'
WHERE t.title='Official transcript'
AND NOT EXISTS (SELECT 1 FROM ticket_feedback f WHERE f.ticket_id=t.ticket_id);

-- 13) Knowledge Base
-----
-- Categories (IT root, Wi-Fi child)
INSERT INTO kb_category(parent_id, name, description)
SELECT NULL, 'IT', 'IT root category'
WHERE NOT EXISTS (SELECT 1 FROM kb_category WHERE parent_id IS NULL AND name='IT');

INSERT INTO kb_category(parent_id, name, description)
SELECT p.category_id, 'Wi-Fi', 'Wireless network issues & guides'
FROM kb_category p
WHERE p.parent_id IS NULL AND p.name='IT'
AND NOT EXISTS (SELECT 1 FROM kb_category c WHERE c.name='Wi-Fi' AND c.parent_id=p.category_id);

-- Articles
INSERT INTO kb_article(category_id, created_by, updated_by, title, content, status)
SELECT c.category_id, u.user_id, u.user_id, 'Connect to Eduroam',
'Step 1: Select eduroam. Step 2: Use Uni ID. Step 3: Accept cert.', 'PUBLISHED'
FROM kb_category c
JOIN [user] u ON u.university_id='E2025-101'
WHERE c.name='Wi-Fi'
AND NOT EXISTS (SELECT 1 FROM kb_article a WHERE a.title='Connect to Eduroam');

INSERT INTO kb_article(category_id, created_by, updated_by, title, content, status)
SELECT c.category_id, u.user_id, u.user_id, 'Reset your password',
'Use password portal → answer security questions → set new password.', 'PUBLISHED'
FROM kb_category c
JOIN [user] u ON u.university_id='E2025-102'
WHERE c.name='IT' AND c.parent_id IS NULL
AND NOT EXISTS (SELECT 1 FROM kb_article a WHERE a.title='Reset your password');

```

```

-- Ratings
INSERT INTO kb_rating(article_id, user_id, rating, comment)
SELECT a.article_id, u.user_id, 4, 'Worked after restarting.'
FROM kb_article a
JOIN [user] u ON u.university_id='U2025-001'
WHERE a.title='Connect to Eduroam'
AND NOT EXISTS (SELECT 1 FROM kb_rating r WHERE r.article_id=a.article_id AND r.user_id=u.user_id);

INSERT INTO kb_rating(article_id, user_id, rating, comment)
SELECT a.article_id, u.user_id, 5, 'Very clear steps.'
FROM kb_article a
JOIN [user] u ON u.university_id='U2025-002'
WHERE a.title='Reset your password'
AND NOT EXISTS (SELECT 1 FROM kb_rating r WHERE r.article_id=a.article_id AND r.user_id=u.user_id);

-----
-- 14) Routing rule / Notification / Settings / Audit
-----

INSERT INTO routing_rule(applyes_to_category, severity_threshold, default_dept_id, default_role_id, active)
SELECT 'Network','HIGH', d.dept_id, r.role_id, 1
FROM department d CROSS JOIN [role] r
WHERE d.name='IT Helpdesk' AND r.name='AGENT'
AND NOT EXISTS (
    SELECT 1 FROM routing_rule rr
    WHERE rr.applies_to_category='Network' AND rr.severity_threshold='HIGH'
);

INSERT INTO notification(user_id, ticket_id, channel, subject, body, delivery_status)
SELECT u.user_id, t.ticket_id, 'EMAIL', 'Ticket Received', 'We are working on your Wi-Fi issue.', 'SENT'
FROM [user] u JOIN ticket t ON t.title='Wi-Fi not working'
WHERE u.university_id='U2025-001'
AND NOT EXISTS (
    SELECT 1 FROM notification n WHERE n.user_id=u.user_id AND n.ticket_id=t.ticket_id AND n.subject='Ticket Received'
);

INSERT INTO system_setting(setting_key, value)
SELECT v.setting_key, v.value
FROM (VALUES ('SLA_HOURS_P1','4'),('SLA_HOURS_P2','8'),('SLA_HOURS_P3','24')) AS v(setting_key, value)
WHERE NOT EXISTS (SELECT 1 FROM system_setting s WHERE s.setting_key=v.setting_key);

INSERT INTO audit_log(actor_id, action, entity_type, entity_id, details)
SELECT u.user_id, 'STATUS_CHANGE','TICKET', t.ticket_id, 'OPEN → IN_PROGRESS'
FROM [user] u JOIN ticket t ON t.title='Wi-Fi not working'
WHERE u.university_id='E2025-101'
AND NOT EXISTS (SELECT 1 FROM audit_log al WHERE al.entity_type='TICKET' AND al.entity_id=t.ticket_id AND al.action='STATUS_CHANGE');

INSERT INTO audit_log(actor_id, action, entity_type, entity_id, details)
SELECT u.user_id, 'RESOLVE','TICKET', t.ticket_id, 'Resolved transcript request'
FROM [user] u JOIN ticket t ON t.title='Official transcript'
WHERE u.university_id='E2025-102'
AND NOT EXISTS (SELECT 1 FROM audit_log al WHERE al.entity_type='TICKET' AND al.entity_id=t.ticket_id AND al.action='RESOLVE');

```


Test Part C

test part C.sql - D:\HQ5FRH3\isu23 (56) Part D.sql - DESKTOP\Q5FRH3\isu23 (75) SQLQuery1.sql - DE...\Q5FRH3\isu23 (75)

```

SELECT * FROM [user] ORDER BY user_id;
SELECT * FROM [role] ORDER BY role_id;
SELECT * FROM department ORDER BY dept_id;

SELECT * FROM student_profile ORDER BY user_id;
SELECT * FROM staff_profile ORDER BY user_id;

```

100 %

Results Messages

	user_id	university_id	full_name	email	password_hash	status	created_at	last_login_at
1	1	U2025-001	Nimal Perera	nimal@uni.test	hash_nimal	ACTIVE	2025-10-09 20:30:23.8077007	NULL
2	2	U2025-002	Sithmi Karun	sithmi@uni.test	hash_sithmi	ACTIVE	2025-10-09 20:30:23.8077007	NULL
3	3	E2025-101	Kasun Silva	kasun@uni.test	hash_kasun	ACTIVE	2025-10-09 20:30:23.8077007	NULL
4	4	E2025-102	Tharuni Dias	tharuni@uni.test	hash_tharuni	ACTIVE	2025-10-09 20:30:23.8077007	NULL
5	5	E2025-001	Manager IT	manager.it@uni.test	hash_mgr	ACTIVE	2025-10-09 20:30:23.8077007	NULL

	role_id	name
1	1	STUDENT
2	2	STAFF
3	3	AGENT
4	4	MANAG...

	dept_id	name	type
1	1	IT Helpdesk	IT
2	2	Academic Affairs	ACADEMIC
3	3	Admin Office	ADMIN

	user_id	program	year_of_study
1	1	BSc IT	2
2	2	BSc CS	1

	user_id	employee_no
1	3	EMP-101
2	4	EMP-102
3	5	EMP-001

	user_id	role_id	assigned_at
1	1	1	2025-10-09 20:31:16.2751032
2	2	1	2025-10-09 20:31:16.2751032
3	3	2	2025-10-09 20:31:16.2751032
4	3	3	2025-10-09 20:31:16.2751032
5	4	2	2025-10-09 20:31:16.2751032
6	4	3	2025-10-09 20:31:16.2751032
7	5	2	2025-10-09 20:31:16.2751032
8	5	4	2025-10-09 20:31:16.2751032

	user_id	dept_id
1	3	1
2	4	1
3	5	1

100 %

Results

Messages

3

5

1

ticket_id	created_by	current_assignee_id	current_dept_id	title	description	category	severity	priority	status	confidential_flag	created_at	updated_at	sla_due_at	resolved_at	resolution_code	feedback_due_at	device_info	location_id
1	1	3	1	NULL	Wi-Fi not working	Network	HIGH	P1	IN_PROGRESS	0	2025-10-09 20:32:15.2680525	2025-10-09 20:32:15.2680525	NULL	NULL	NULL	NULL	Laptop - Dell	Library L1
2	1	NULL	1	Lab PC broken	PC in Lab-3 not booting	Hardware	HIGH	P1	ON_HOLD	0	2025-10-09 20:32:15.2680525	2025-10-09 20:32:15.2680525	NULL	NULL	NULL	NULL	Desktop - Lab 3	Lab 3
3	2	NULL	1	Password reset	Forgot LMS password, need reset	Academy	MED	P2	OPEN	0	2025-10-09 20:32:15.2680525	2025-10-09 20:32:15.2680525	NULL	NULL	NULL	NULL	NULL	NULL
4	2	NULL	2	Official transcript	Need official transcript for internship	Academy	LOW	P3	RESOLVED	0	2025-10-09 20:32:15.2680525	2025-10-09 20:32:15.2680525	NULL	NULL	NULL	NULL	NULL	NULL
5	1	NULL	1	Printer toner low	Lab-2 printer shows toner low	Network	HIGH	P2	OPEN	0	2025-10-09 21:03:39.3970721	2025-10-09 21:18:01.0090012	NULL	NULL	NULL	NULL	NULL	NULL

message_id	ticket_id	sender_id	visibility	body	created_at	
3	3	3	2	PUBLIC	Forgot password, registered email is sdtm@unl...	2025-10-09 20:32:31.2049908
4	4	2	1	PUBLIC	PC says "No boot device"	2025-10-09 20:32:31.2049908
5	5	4	4	PUBLIC	Your transcript has been generated and emailed.	2025-10-09 20:32:31.2049908
6	5	1	1	PUBLIC	Trigger test message	2025-10-09 21:12:21.3826239
7	7	5	1	PUBLIC	Trigger test message	2025-10-09 21:12:37.7917578
8	8	5	1	PUBLIC	Trigger test message	2025-10-09 21:16:21.7210887
9	9	5	1	PUBLIC	Trigger test message	2025-10-09 21:17:55.7128577
10	10	5	1	PUBLIC	Trigger test message	2025-10-09 21:17:56.8356437

attachment_id	ticket_id	uploaded_by	file_name	mime_type	file_size	storage_url	uploaded_at
1	1	1	web_screenshot.png	image/png	123456	blob/ticketid11web_screenshot.png	2025-10-09 20:32:44.2587354

attachment_id	message_id	uploaded_by	file_name	mime_type	file_size	storage_url	uploaded_at
---------------	------------	-------------	-----------	-----------	-----------	-------------	-------------

history_id	ticket_id	from_status	to_status	changed_by	note	changed_at
1	1	OPEN	IN_PROGRESS	3	Picked up by agent1	2025-10-09 20:33:08.3992720
2	2	OPEN	IN_PROGRESS	4	Assigned to agent2	2025-10-09 20:33:08.4150184
3	3	IN_PROG...	RESOLVED	4	Completed and no...	2025-10-09 20:33:08.4400370

assign_id	ticket_id	assignee_user_id	assignee_dept_id	assignment_type	assigned_by	note	is_active	assigned_at	
1	1	3	1	DIRECT	5	Assigned to agent1	1	2025-10-09 20:33:39.8306929	
2	3	NULL	1	AUTO	5	Auto-routed by rule	1	2025-10-09 20:33:57.7596015	
3	3	2	NULL	1	AUTO	5	Initial routing	0	2025-10-09 20:34:17.3283390
4	4	2	4	NULL	ESCALATION	5	Escalated to age...	1	2025-10-09 20:34:58.0581094
5	5	4	4	NULL	DIRECT	5	Handled and res...	0	2025-10-09 20:38:08.7040799
6	9	5	NULL	1	DIRECT	5	NULL	0	2025-10-09 21:17:50.7272994
7	10	5	NULL	1	DIRECT	5	NULL	1	2025-10-09 21:18:00.8666796

link_id	primary_ticket_id	related_ticket_id	link_type	linked_by	linked_at	
1	1	1	2	RELATED	3	2025-10-09 20:39:39.0548754

hold_id	ticket_id	sent_by	reason	reminder_at	released_at	
1	1	2	4	Waiting for spare parts	2025-10-11 20:39:56.5411389	NULL

feedback_id	ticket_id	student_id	rating	comment	submitted_at	
1	1	4	2	5	Quick and helpful!	2025-10-09 20:39:56.5411389

category_id	parent_id	name	description
1	NULL	IT	IT root category
2	1	Wi-Fi	Wireless network issues & guides

article_id	category_id	created_by	updated_by	title	content	status	view_count	created_at	updated_at	
1	1	2	3	3	Connect to Eduroam	Step 1: Select eduroam. Step 2: Use Uni ID. Step...	PUBLISHED	0	2025-10-09 20:40:43.1959303	2025-10-09 20:40:43.1959303
2	2	1	4	4	Reset your password	Use password portal ? answer security questions...	PUBLISHED	0	2025-10-09 20:40:43.1959303	2025-10-09 20:40:43.1959303

rating_id	article_id	user_id	rating	comment	created_at	
1	1	1	1	4	Worked after restarting.	2025-10-09 20:40:57.9187299
2	2	2	2	5	Very clear steps.	2025-10-09 20:40:57.9238537

rule_id	applies_to_category	severity_threshold	default_dept_id	default_role_id	active
---------	---------------------	--------------------	-----------------	-----------------	--------

notification_id	user_id	ticket_id	channel	subject	body	delivery_status	sent_at
1	1	1	EMAIL	Ticket Received	We are working on your Wi-Fi issue.	SENT	2025-10-09 20:41:36.9462840

setting_key	value	updated_at	
1	SLA_HOURS_P1	4	2025-10-09 20:41:41.1665805
2	SLA_HOURS_P2	8	2025-10-09 20:41:41.1665805
3	SLA_HOURS_P3	24	2025-10-09 20:41:41.1665805

log_id	actor_id	action	entity_type	entity_id	details	logged_at	
1	1	3	STATUS_CHANGE	TICKET	1	OPEN ? IN_PROGRESS	2025-10-09 20:41:45.9418438
2	2	4	RESOLVE	TICKET	4	Resolved transcript request	2025-10-09 20:41:51.0548724

4. Part D – SQL Queries & Outputs

```
Part D.sql - DESK...Q5FRH3\isu23 (75))  SQLQuery1.sql - DE...Q5FRH3\isu23 (64))*  Part C.sql - DESK...Q5FRH3\isu23
-- 1. Tickets overview
SELECT
    t.ticket_id,
    t.title,
    creator.full_name      AS created_by,
    assignee.full_name     AS current_assignee,
    d.name                 AS current_dept,
    t.severity, t.priority, t.status,
    t.created_at
FROM ticket t
JOIN [user] creator      ON creator.user_id = t.created_by
LEFT JOIN [user] assignee ON assignee.user_id = t.current_assignee_id
LEFT JOIN department d   ON d.dept_id = t.current_dept_id
ORDER BY t.ticket_id;

-- 2. Conversation for "Wi-Fi not working"
SELECT t.title, m.message_id, u.full_name AS sender, m.visibility, m.body, m.created_at
FROM ticket_message m
JOIN ticket t ON t.ticket_id = m.ticket_id
JOIN [user] u ON u.user_id = m.sender_id
WHERE t.title = 'Wi-Fi not working'
ORDER BY m.created_at, m.message_id;

-- 3. Ticket count by status
SELECT status, COUNT(*) AS total
FROM ticket
GROUP BY status
ORDER BY total DESC, status;

--4. Active assignments
SELECT
    t.title,
    CASE WHEN a.assignee_user_id IS NOT NULL THEN 'USER' ELSE 'DEPT' END AS assigned_to_type,
    COALESCE(u.full_name, d.name) AS assigned_to,
    a.assignment_type, a.assigned_at
FROM ticket_assignment a
JOIN ticket t          ON t.ticket_id = a.ticket_id
LEFT JOIN [user] u      ON u.user_id = a.assignee_user_id
LEFT JOIN department d  ON d.dept_id = a.assignee_dept_id
WHERE a.is_active = 1
ORDER BY t.title;
```

ticket_id	title	created_by	current_assignee	current_dept	severity	priority	status	created_at
1	Wi-Fi not working	Nimal Perera	Kasun Silva	NULL	HIGH	P1	IN_PROGRESS	2025-10-09 20:32:15.2680525
2	Lab PC broken	Nimal Perera	NULL	IT Helpdesk	HIGH	P1	ON_HOLD	2025-10-09 20:32:15.2680525
3	Password reset	Sithmi Karun	NULL	IT Helpdesk	MEDIUM	P2	OPEN	2025-10-09 20:32:15.2680525
4	Official transcript	Sithmi Karun	NULL	Academic Affairs	LOW	P3	RESOLVED	2025-10-09 20:32:15.2680525
5	Printer toner low	Nimal Perera	NULL	IT Helpdesk	HIGH	P2	OPEN	2025-10-09 21:03:39.3970721
6	Printer toner low	Nimal Perera	NULL	NULL	HIGH	P2	OPEN	2025-10-10 06:03:11.5004991

title	message_id	sender	visibility	body	created_at
Wi-Fi not working	1	Nimal Perera	PUBLIC	I tried eduroam and the guest network, neither w...	2025-10-09 20:32:31.2049908
Wi-Fi not working	2	Kasun Silva	INTERNAL	Checking AP status near Library L1.	2025-10-09 20:32:31.2049908

status	total
OPEN	3
IN_PROGRESS	1
ON_HOLD	1
RESOLVED	1

title	assigned_to_type	assigned_to	assignment_type	assigned_at
Lab PC broken	USER	Tharuni Dias	ESCALATION	2025-10-09 20:34:58.0581094
Password reset	DEPT	IT Helpdesk	AUTO	2025-10-09 20:33:57.7596015
Printer toner low	DEPT	IT Helpdesk	DIRECT	2025-10-09 21:18:00.9666796
Wi-Fi not working	USER	Kasun Silva	DIRECT	2025-10-09 20:33:39.9306929

5. Part E – Stored Function/Procedure

```

Part E.sql - DESK...HQ5FRH3\isu23 (77)*  X  Part D.sql - DESK...Q5FRH3\isu23 (75)  SQLQuery1.sql - DE...Q5FRH3\isu23 (64)*
/* =====
PART E – Stored Functions & Procedures
===== */

-- 1) Helper scalar functions
-----
GO
CREATE OR ALTER FUNCTION dbo.fn_SeverityRank (@severity VARCHAR(10))
RETURNS TINYINT
AS
BEGIN
    RETURN CASE UPPER(@severity)
        WHEN 'LOW' THEN 1
        WHEN 'MEDIUM' THEN 2
        WHEN 'HIGH' THEN 3
        WHEN 'CRITICAL' THEN 4
        ELSE 0
    END;
END;
GO

CREATE OR ALTER FUNCTION dbo.fn_IsStudent (@user_id INT)
RETURNS BIT
AS
BEGIN
    RETURN CASE WHEN EXISTS (SELECT 1 FROM student_profile WHERE user_id = @user_id) THEN 1 ELSE 0 END;
END;
GO

CREATE OR ALTER FUNCTION dbo.fn_TicketIsOverdue (@ticket_id INT)
RETURNS BIT
AS
BEGIN
    DECLARE @due DATETIME2 = (SELECT sla_due_at FROM ticket WHERE ticket_id=@ticket_id);
    IF @due IS NULL RETURN 0;

    DECLARE @resolved DATETIME2 = (SELECT resolved_at FROM ticket WHERE ticket_id=@ticket_id);
    IF @resolved IS NOT NULL
        RETURN CASE WHEN @resolved > @due THEN 1 ELSE 0 END;

    RETURN CASE WHEN SYSDATETIME() > @due THEN 1 ELSE 0 END;
END;
GO

```

```

-----
-- 2) Ticket summary (table-valued view)
-----
GO
CREATE OR ALTER FUNCTION dbo.fn_TicketSummary (@ticket_id INT)
RETURNS TABLE
AS
RETURN
SELECT
    t.ticket_id, t.title, t.category, t.severity, t.priority, t.status,
    t.created_at, t.updated_at, t.sla_due_at, t.resolved_at,
    dbo.fn_TicketIsOverdue(t.ticket_id) AS is_overdue,
    u.full_name AS created_by_name, u.email AS created_by_email,
    ua.full_name AS current_assignee_name,
    d.name AS current_dept_name
FROM ticket t
JOIN [user] u ON u.user_id = t.created_by
LEFT JOIN [user] ua ON ua.user_id = t.current_assignee_id
LEFT JOIN department d ON d.dept_id = t.current_dept_id
WHERE t.ticket_id = @ticket_id;
GO

-----
-- 3) Core Stored Procedures (SPs)
-----
GO
CREATE OR ALTER PROCEDURE dbo.usp_CreateTicket
    @created_by_university_id VARCHAR(20),
    @title VARCHAR(200),
    @description VARCHAR(MAX),
    @category VARCHAR(50),
    @severity VARCHAR(10),
    @priority VARCHAR(5),
    @status VARCHAR(20) = 'OPEN',
    @confidential_flag BIT = 0,
    @device_info VARCHAR(200) = NULL,
    @location_text VARCHAR(200) = NULL,
    @assignee_university_id VARCHAR(20) = NULL,
    @assignee_dept_name VARCHAR(100) = NULL
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    BEGIN TRY
        BEGIN TRAN;

        DECLARE @creator INT = (SELECT user_id FROM [user] WHERE university_id=@created_by_university_id);
        IF @creator IS NULL THROW 50000, 'Creator not found.', 1;

        IF dbo.fn_SeverityRank(@severity)=0 THROW 50000, 'Invalid severity.', 1;
        IF @priority NOT IN ('P1','P2','P3') THROW 50000, 'Invalid priority.', 1;
        IF @status NOT IN ('OPEN','IN_PROGRESS','ON_HOLD','RESOLVED','CLOSED') THROW 50000, 'Invalid status.', 1;

        INSERT INTO ticket(created_by, current_assignee_id, current_dept_id, title, description, category,
            severity, priority, status, confidential_flag, device_info, location_text)
        VALUES (@creator, NULL, NULL, @title, @description, @category, @severity, @priority, @status, @confidential_flag, @device_info, @location_text);

        DECLARE @ticket_id INT = SCOPE_IDENTITY();
    END TRY
    BEGIN CATCH
        ROLLBACK;
    END CATCH
END

```

```

-- Optional initial assignment
IF @assignee_university_id IS NOT NULL OR @assignee_dept_name IS NOT NULL
BEGIN
    EXEC dbo.usp_AssignTicket
        @ticket_id=@ticket_id,
        @assignee_university_id=@assignee_university_id,
        @assignee_dept_name=@assignee_dept_name,
        @assignment_type='DIRECT',
        @assigned_by_university_id=@created_by_university_id,
        @note='Initial assignment',
        @make_active=1;

    END

    SELECT * FROM dbo.fn_TicketSummary(@ticket_id);
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT>0 ROLLBACK;
    THROW;
END CATCH
END;
GO

/* Add a message to a ticket (PUBLIC or INTERNAL) */
GO
CREATE OR ALTER PROCEDURE dbo.usp_AddTicketMessage
    @ticket_id INT,
    @sender_university_id VARCHAR(20),
    @visibility VARCHAR(10),
    @body VARCHAR(MAX)
AS
BEGIN
    SET NOCOUNT ON;
    IF @visibility NOT IN ('PUBLIC', 'INTERNAL') THROW 50000, 'Invalid visibility.', 1;

    DECLARE @sender INT = (SELECT user_id FROM [user] WHERE university_id=@sender_university_id);
    IF @sender IS NULL THROW 50000, 'Sender not found.', 1;

    IF NOT EXISTS (SELECT 1 FROM ticket WHERE ticket_id=@ticket_id) THROW 50000, 'Ticket not found.', 1;

    INSERT INTO ticket_message(ticket_id, sender_id, visibility, body)
    VALUES (@ticket_id, @sender, @visibility, @body);

    -- touch ticket updated_at
    UPDATE ticket SET updated_at = SYSDATETIME() WHERE ticket_id=@ticket_id;

    SELECT SCOPE_IDENTITY() AS message_id;
END;
GO

```

```

/* Change ticket status and record history */
GO
CREATE OR ALTER PROCEDURE dbo.usp_ChangeTicketStatus
    @ticket_id INT,
    @new_status VARCHAR(20),
    @changed_by_university_id VARCHAR(20),
    @note VARCHAR(MAX) = NULL
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    IF @new_status NOT IN ('OPEN', 'IN_PROGRESS', 'ON_HOLD', 'RESOLVED', 'CLOSED') THROW 50000, 'Invalid status.', 1;

    DECLARE @changed_by INT = (SELECT user_id FROM [user] WHERE university_id=@changed_by_university_id);
    IF @changed_by IS NULL THROW 50000, 'User not found.', 1;

    BEGIN TRY
        BEGIN TRAN;

        DECLARE @old_status VARCHAR(20) = (SELECT status FROM ticket WHERE ticket_id=@ticket_id);
        IF @old_status IS NULL THROW 50000, 'Ticket not found.', 1;
        IF @old_status = @new_status RETURN; -- no-op

        UPDATE ticket
            SET status=@new_status,
                updated_at=SYSDATETIME(),
                resolved_at = CASE WHEN @new_status IN ('RESOLVED', 'CLOSED') THEN COALESCE(resolved_at, SYSDATETIME()) ELSE resolved_at END
        WHERE ticket_id=@ticket_id;

        INSERT INTO ticket_status_history(ticket_id, from_status, to_status, changed_by, note)
            VALUES (@ticket_id, @old_status, @new_status, @changed_by, @note);

        COMMIT;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT>0 ROLLBACK;
        THROW;
    END CATCH
END;
GO

```

```

/* Assign (or reassign) a ticket.
Enforces XOR (user OR dept) and ensures only one active assignment.
Also updates the ticket.current_assignee_id / current_dept_id.
*/
GO
CREATE OR ALTER PROCEDURE dbo.usp_AssignTicket
    @ticket_id INT,
    @assignee_university_id VARCHAR(20) = NULL,
    @assignee_dept_name VARCHAR(100) = NULL,
    @assignment_type VARCHAR(15),
    @assigned_by_university_id VARCHAR(20),
    @note VARCHAR(MAX) = NULL,
    @make_active BIT = 1
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;

    IF @assignment_type NOT IN ('DIRECT', 'AUTO', 'ESCALATION') THROW 50000, 'Invalid assignment_type.', 1;
    IF (@assignee_university_id IS NULL AND @assignee_dept_name IS NULL) OR
        (@assignee_university_id IS NOT NULL AND @assignee_dept_name IS NOT NULL)
        THROW 50000, 'Provide either assignee user OR department (not both).', 1;

    DECLARE @assigned_by INT = (SELECT user_id FROM [user] WHERE university_id=@assigned_by_university_id);
    IF @assigned_by IS NULL THROW 50000, 'Assigned-by user not found.', 1;

    DECLARE @assignee_user_id INT = NULL, @assignee_dept_id INT = NULL;

    IF @assignee_university_id IS NOT NULL
        SET @assignee_user_id = (SELECT user_id FROM [user] WHERE university_id=@assignee_university_id);

    IF @assignee_dept_name IS NOT NULL
        SET @assignee_dept_id = (SELECT dept_id FROM department WHERE name=@assignee_dept_name);

```



```

IF @assignee_university_id IS NOT NULL AND @assignee_user_id IS NULL
    THROW 50000, 'Assignee user not found.', 1;

IF @assignee_dept_name IS NOT NULL AND @assignee_dept_id IS NULL
    THROW 50000, 'Assignee department not found.', 1;

IF NOT EXISTS (SELECT 1 FROM ticket WHERE ticket_id=@ticket_id) THROW 50000, 'Ticket not found.', 1;

BEGIN TRY
    BEGIN TRAN;

    IF @make_active = 1
    BEGIN
        -- deactivate any existing active assignment
        UPDATE ticket_assignment
            SET is_active = 0
            WHERE ticket_id=@ticket_id AND is_active=1;
    END

    INSERT INTO ticket_assignment(ticket_id, assignee_user_id, assignee_dept_id, assignment_type, assigned_by, note, is_active)
    VALUES (@ticket_id, @assignee_user_id, @assignee_dept_id, @assignment_type, @assigned_by, @note, @make_active);

    -- reflect on ticket
    IF @make_active = 1
    BEGIN
        UPDATE ticket
            SET current_assignee_id = @assignee_user_id,
                current_dept_id = @assignee_dept_id,
                updated_at = SYSDATETIME()
            WHERE ticket_id=@ticket_id;
    END

    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT>0 ROLLBACK;
    THROW;
END CATCH
END;
GO

/* Link two tickets (orders the pair so duplicates are avoided) */
GO

CREATE OR ALTER PROCEDURE dbo.usp_LinkTickets
    @ticket_id_a INT,
    @ticket_id_b INT,
    @link_type VARCHAR(20),
    @linked_by_university_id VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;
    IF @ticket_id_a = @ticket_id_b THROW 50000, 'Cannot link a ticket to itself.', 1;

    DECLARE @linked_by INT = (SELECT user_id FROM [user] WHERE university_id=@linked_by_university_id);
    IF @linked_by IS NULL THROW 50000, 'Linking user not found.', 1;

    DECLARE @p INT = CASE WHEN @ticket_id_a < @ticket_id_b THEN @ticket_id_a ELSE @ticket_id_b END;
    DECLARE @r INT = CASE WHEN @ticket_id_a < @ticket_id_b THEN @ticket_id_b ELSE @ticket_id_a END;

    IF NOT EXISTS (SELECT 1 FROM ticket WHERE ticket_id IN (@p, @r)) THROW 50000, 'Ticket not found.', 1;

    IF NOT EXISTS (SELECT 1 FROM ticket_link WHERE primary_ticket_id=@p AND related_ticket_id=@r)
    INSERT INTO ticket_link(primary_ticket_id, related_ticket_id, link_type, linked_by)
    VALUES (@p, @r, @link_type, @linked_by);
END;
GO

```

```

/* Add student feedback (enforces: one per ticket and user must be student) */
GO
CREATE OR ALTER PROCEDURE dbo.usp_AddTicketFeedback
    @ticket_id INT,
    @student_university_id VARCHAR(20),
    @rating INT,
    @comment VARCHAR(MAX) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM ticket WHERE ticket_id=@ticket_id) THROW 50000, 'Ticket not found.', 1;

    DECLARE @student_id INT = (SELECT user_id FROM [user] WHERE university_id=@student_university_id);
    IF @student_id IS NULL THROW 50000, 'Student user not found.', 1;
    IF dbo.fn_IsStudent(@student_id) = 0 THROW 50000, 'Only students can submit feedback.', 1;

    IF EXISTS (SELECT 1 FROM ticket_feedback WHERE ticket_id=@ticket_id)
        THROW 50000, 'Feedback already exists for this ticket.', 1;

    IF @rating < 1 OR @rating > 5 THROW 50000, 'Rating must be between 1 and 5.', 1;

    INSERT INTO ticket_feedback(ticket_id, student_id, rating, comment)
    VALUES (@ticket_id, @student_id, @rating, @comment);
END;
GO

/* Auto-route a ticket using routing_rule.
Picks an active rule where applies_to_category = ticket.category
AND rule.severity_threshold <= ticket.severity (by rank).
Then assigns to rule.default_dept_id as ACTIVE.
*/
GO
CREATE OR ALTER PROCEDURE dbo.usp_AutoRouteTicket
    @ticket_id INT,
    @assigned_by_university_id VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;

    DECLARE @cat VARCHAR(50), @sev VARCHAR(10);
    SELECT @cat = category, @sev = severity FROM ticket WHERE ticket_id=@ticket_id;
    IF @cat IS NULL THROW 50000, 'Ticket not found.', 1;

    DECLARE @sevRank TINYINT = dbo.fn_SeverityRank(@sev);

    DECLARE @dept_id INT =
    (
        SELECT TOP(1) rr.default_dept_id
        FROM routing_rule rr
        WHERE rr.active = 1
            AND rr.applies_to_category = @cat
            AND dbo.fn_SeverityRank(rr.severity_threshold) <= @sevRank
        ORDER BY dbo.fn_SeverityRank(rr.severity_threshold) DESC
    );

    IF @dept_id IS NULL THROW 50000, 'No routing rule matched this ticket.', 1;

    DECLARE @dept_name VARCHAR(100) = (SELECT name FROM department WHERE dept_id=@dept_id);

    EXEC dbo.usp_AssignTicket
        @ticket_id=@ticket_id,
        @assignee_university_id=NULL,
        @assignee_dept_name=@dept_name,
        @assignment_type='AUTO',
        @assigned_by_university_id=@assigned_by_university_id,
        @note='Auto-routed by rule',
        @make_active=1;
END;
GO

```


6. Part F – Trigger

```
Part F.sql - DESK...HQ5FRH3\isu23 (57))* X Part E.sql - DESK...HQ5FRH3\isu23 (77))* Part D.sql - DESK...0
/* =====
PART F – Triggers (SQL Server / T-SQL)
===== */

GO

CREATE OR ALTER TRIGGER dbo.trg_ticket_assignment_enforce_and_sync
ON dbo.ticket_assignment
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Collect affected ticket_ids so we can reuse them in multiple statements
    DECLARE @aff TABLE (ticket_id INT PRIMARY KEY);
    INSERT INTO @aff(ticket_id)
    SELECT DISTINCT ticket_id FROM inserted;

    /* If any newly inserted/updated row is ACTIVE, deactivate other active rows
    for the same ticket (enforces the ≤1 active rule). */
    UPDATE ta
    SET is_active = 0
    FROM dbo.ticket_assignment AS ta
    JOIN inserted AS i
    ON ta.ticket_id = i.ticket_id
    AND i.is_active = 1
    AND ta.is_active = 1
    AND ta.assign_id <> i.assign_id;

    -- Final guard: no ticket may have more than one active assignment
    IF EXISTS (
        SELECT ta.ticket_id
        FROM dbo.ticket_assignment AS ta
        JOIN @aff AS a ON a.ticket_id = ta.ticket_id
        WHERE ta.is_active = 1
        GROUP BY ta.ticket_id
        HAVING COUNT(*) > 1
    )
    BEGIN
        RAISERROR('Only one active assignment per ticket is allowed.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END
END
```

```

/* Sync ticket.current_* to the ACTIVE assignment (or NULL if none).
   OUTER APPLY gives us one row per ticket with the active assignee, if any. */
UPDATE t
    SET t.current_assignee_id = ca.assignee_user_id,
        t.current_dept_id     = ca.assignee_dept_id,
        t.updated_at          = SYSDATETIME()
FROM dbo.ticket AS t
JOIN @aff AS a
    ON a.ticket_id = t.ticket_id
OUTER APPLY (
    SELECT TOP (1) ta.assignee_user_id, ta.assignee_dept_id
    FROM dbo.ticket_assignment AS ta
    WHERE ta.ticket_id = t.ticket_id AND ta.is_active = 1
    ORDER BY ta.assign_id DESC
) AS ca;
END;
GO

```

100 %

	current_assignee_id	current_dept_id
1	NULL	1